

Pulso TransMi

Clonado del SDK, extracción de datos, análisis exploratorio y recomendaciones de modelado para el reto MLOps de pronóstico de demanda.

Repositorio	github.com/uexternadojz/pulso-transmi-sdk
API de lectura	pulso-transmi.72-60-245-2.sslip.io
Dataset	pulso-transmi-starter-v1 — 12 estaciones, 51.840 observaciones, 45 días
Rango temporal	2026-07-26 00:00 a 2026-09-08 23:45 (zona -05:00)
Entregable principal	examples/03_ed.ipynb — notebook ejecutado, 48 celdas, 12 gráficos
Entorno	Python 3.12 en .venv (el proyecto exige ≥ 3.11)

Este documento recoge, en orden, lo que se pidió en la conversación y lo que se produjo en cada paso, junto con los hallazgos técnicos que salieron del análisis y las correcciones aplicadas sobre la marcha. Está pensado como constancia de trabajo: cualquiera que lo lea debería poder reconstruir qué se hizo, con qué evidencia y qué quedó pendiente.

01 Cronología de la conversación

Doce solicitudes en total. La columna de resultado apunta al archivo o al hallazgo concreto que quedó de cada una.

#	Lo que se pidió	Lo que se hizo	Resultado
1	Clonar el repositorio del SDK.	git clone sobre el directorio de trabajo.	Proyecto Python con src/, tests/, docs/, examples/ y templates/.
2	Extraer datos de la API.	Se creó el entorno virtual y se ejecutó examples/01_download.py. Python 3.10 del sistema no servía: el proyecto exige 3.11 o superior, así que se rehízo el venv con Python 3.12.	Cuatro archivos en data/: stations.csv, observations.csv, context.csv y metadata.json.
3	Leer el README.	Resumen del reto: pronóstico cada 15 minutos, métrica WAPE promediada por estación, obligación de trazabilidad y de partición temporal.	Contexto para decidir el enfoque del análisis.
4	Crear un script para el EDA.	Se escribió examples/03_eda.py siguiendo la convención numerada de la carpeta.	Reporte de texto en consola más cinco gráficos guardados en disco.
5	Convertirlo en notebook con imágenes y comentarios en línea.	Se reconstruyó como examples/03_eda.ipynb y se ejecutó con nbconvert para dejar las salidas embebidas. Los comentarios pasaron a celdas markdown.	Notebook navegable sin necesidad de ejecutarlo. Se eliminó la versión .py.
6	Añadir un gráfico geográfico.	Nueva sección con dos mapas sobre coordenadas reales y un análisis de picos AM/PM.	Hallazgo: la geografía define el rol de la estación, no su tamaño.
7	Añadir también uno de correlación.	Nueva sección con matriz de correlación y comparación agrupada frente a intra-estación.	Hallazgo: la temperatura es el ciclo horario disfrazado.
8	Los boxplots muestran números, no nombres de estación.	Las etiquetas usaban station_id ('07111'). Se cambiaron a station_name en boxplot, perfil horario y tabla de autocorrelación.	Corregido y notebook re-ejecutado.
9	Crear un artifact con el EDA, datos faltantes, mapa, correlaciones y modelos sugeridos.	Se exportaron todos los datos del EDA a JSON y se lanzaron dos workflows con 17 agentes en total para investigar modelos y auditar vacíos de datos.	Parcialmente completado: ver sección 05.
10	Sugerencia de diseño para las tablas del proyecto.	Se consultó a qué se refería y se confirmó: esquema de base de datos. Se lanzó un workflow dedicado con tres diseños independientes y crítica adversaria.	Tres diseños completos, síntesis final no alcanzada.
11	Solo el esquema, no subirlo a Supabase.	Confirmado. El trabajo quedó en modo diseño; no se ejecutó nada contra ninguna instancia.	Sin efectos sobre sistemas externos.
12	Un PDF con la conversación.	Este documento.	Conversacion-Pulso-TransMi.pdf

02 Hallazgos del análisis exploratorio

El notebook cubre ocho secciones. Estos son los resultados que cambian decisiones de modelado, no el inventario completo de estadísticas.

Integridad: el dataset histórico es perfecto

Cero huecos temporales, cero duplicados, cero nulos, cero valores negativos y ningún cero en la demanda. Las 12 estaciones tienen exactamente los 4.320 periodos esperados. Los hashes SHA-256 de los tres CSV coinciden con los declarados en metadata.json.

La consecuencia práctica es que no hace falta imputar nada, y por tanto cualquier valor nulo que aparezca más adelante en el pipeline es un error propio —típicamente lags mal construidos— y no un defecto del origen.

Demanda por estación: varía tres veces entre extremos

Estación	Media	Mediana	CV	Pico AM/PM	Rol
Ricaurte - NQS	683,7	452	0,67	0,97	Transferencia
Banderas	591,1	392	0,66	1,01	Transferencia
Portal El Dorado	510,9	340	0,67	1,03	Transferencia
Portal Américas	342,1	201	0,84	1,40	Origen
Museo Nacional	337,9	266	0,80	0,23	Destino
Calle 100	294,0	158	0,95	0,66	Destino
Universidad Nacional	281,0	248	0,71	1,38	Origen
Movistar Arena	272,6	213	0,80	0,20	Destino
Portal Suba	258,4	153	0,84	1,76	Origen
Calle 72	249,8	134	0,94	0,74	Destino
Universidades - CityU	238,9	207	0,71	1,31	Origen
Portal Usme	217,9	128	0,84	1,26	Origen

En todas las estaciones la mediana queda muy por debajo de la media: son distribuciones sesgadas a la derecha, con muchos periodos de demanda baja y picos pronunciados. Esto importa porque la métrica del reto promedia el WAPE por estación, así que Portal Usme pesa lo mismo que Ricaurte pese a tener un tercio de su demanda.

Patrones temporales

- Doble pico diario: máximo de 677 hacia las 7am y de 701 hacia las 5pm, con valle nocturno cercano a 140.
- El fin de semana la demanda cae 21,8% respecto a los días laborales (297,6 frente a 380,5).
- La serie diaria es estable en nivel: la media móvil de 7 días apenas se mueve y la oscilación visible es el ciclo semanal, no una tendencia.

Geografía: define el rol, no el tamaño

Comparando el pico de la mañana (6–8h) con el de la tarde (16–18h) aparece una estructura espacial clara. Los portales periféricos son estaciones de origen —la gente sale de ahí por la mañana— y las centrales son de destino.

Medida	Valor	Lectura
corr(distancia al centro, cociente AM/PM)	0,507	El rol se ordena espacialmente
corr(distancia al centro, demanda media)	-0,189	La distancia no predice el volumen
Cociente AM/PM medio en portales	1,364	Origen
Cociente AM/PM medio fuera de portales	0,811	Destino

Las excepciones son coherentes: Universidad Nacional (1,38) y Universidades–CityU (1,31) son céntricas pero se comportan como origen, porque son campus con llegada matutina concentrada. Movistar Arena es el extremo opuesto (0,20): un recinto de eventos, con casi toda su demanda por la tarde.

Autocorrelación: el lag semanal gana al diario

Lag	Periodos	Autocorrelación media	Comentario
1 semana	672	0,943	El más alto y el más estable entre estaciones
15 minutos	1	0,940	Alto pero inútil a horizontes largos
1 día	96	0,890	Es el que usa el baseline oficial del SDK
1 hora	4	0,790	Cae por la pendiente de las horas punta

El hallazgo más accionable

El baseline oficial de `examples/02_naive_baseline.py` usa `shift(96)`, es decir el lag de un día. El lag de una semana tiene mayor autocorrelación (0,943 frente a 0,890) y además es mucho más estable: varía solo entre 0,925 y 0,959 en las 12 estaciones, mientras el diario cae hasta 0,788 en las estaciones de campus. Cambiar `shift(96)` por `shift(672)` debería mejorar el baseline sin ningún modelo de por medio.

Contexto: clima y eventos aportan poco, y esconden trampas

Variable	Correlación con demanda	Lectura
temperature_forecast	0,339	Engañosa: ver abajo
event_intensity	0,145	Solo sirve umbralizada
rain_forecast	-0,013	Sin señal utilizable

La temperatura correlaciona **-0,878 con el coseno de la hora**. Es decir, no mide un efecto del clima sobre la demanda: es el ciclo diario entrando por otra vía. Con features de hora ya en el modelo, la temperatura aporta poco y añade redundancia.

En cambio los pronósticos sí son fieles al valor real —0,982 en temperatura y 0,823 en lluvia—, así que pueden usarse como features sin perder casi información. Y deben usarse esos: al predecir el futuro solo existe el pronóstico, de modo que emplear `rain_mm` o `temperature_c` sería fuga de información.

La correlación lineal engaña dos veces

- **Subestima la hora.** Las features de hora tienen correlación de Pearson bajísima (-0,09 y -0,19) pese a que el perfil horario es el patrón más fuerte del dataset. La causa es que el perfil es bimodal —sube, baja, vuelve a subir— y Pearson solo mide relaciones lineales. Descartar la hora por su correlación sería un error grave.
- **Infla lo que solo es escala.** La media móvil de un día pasa de 0,473 agrupando todas las estaciones a 0,144 calculada dentro de cada una: la mayor parte de esa señal era simplemente que las estaciones grandes tienen medias grandes. Los lags, en cambio, apenas cambian (0,953 a 0,943), lo que confirma que su señal sí es temporal.

03 Correcciones aplicadas durante la sesión

Dos errores se detectaron y corrigieron sobre la marcha. Se dejan documentados porque ambos afectaban conclusiones, no solo presentación.

Error propio: la proporción de periodos con evento

En una versión anterior del notebook se afirmó que «solo ~3% de los periodos tienen evento activo». La cifra salía de confundir la *media* de la columna (0,032) con la *proporción* de periodos no nulos. Verificado correctamente, el reparto real es otro:

Umbral	Proporción de periodos	Utilidad como bandera
mayor que 0	89,5%	Inútil: marcaría casi todo el dataset
mayor que 0,01	7,8%	Razonable
mayor que 0,1	5,5%	Razonable
mayor que 0,5	3,0%	Solo eventos fuertes
exactamente 0	10,5%	—

El 89,5% no significa que haya eventos constantes: el generador deja valores denormales minúsculos, siendo el menor no nulo del orden de $1e-323$. La lección práctica es que `event_intensity > 0` no sirve como variable booleana; hay que umbralizar. El notebook ya refleja la versión corregida.

Defecto del entorno: pandas y la alineación de índices

Al construir la media móvil por estación, una versión intermedia usó `reset_index(level=0, drop=True)` sobre un índice que no era jerárquico, lo que desalineó silenciosamente la serie con el resto del DataFrame y produjo una correlación falsa de 0,04. Se corrigió con `groupby.transform`, que preserva la alineación. El valor correcto es 0,473. No hubo error ni advertencia: el resultado simplemente era falso.

04 Estructura del notebook entregado

Sección	Contenido	Salidas
Carga de datos	Lectura con tipos correctos y derivación de hora, día y fecha	3 tablas
1. Integridad	Huecos, duplicados, nulos, negativos	Reporte
2. Demanda por estación	Estadísticos y dispersión	Tabla + boxplot
3. Distribución geográfica	Mapas por corredor y por rol origen/destino	3 gráficos + tabla
4. Patrones temporales	Perfil horario, día de semana, mapa día×hora	4 gráficos
5. Deriva temporal	Serie diaria y comparación últimos 7 días	Gráfico + tabla
6. Autocorrelación	Lags de 15 min, 1 hora, 1 día y 1 semana	Tabla
7. Contexto	Clima, eventos, fidelidad del pronóstico	Gráfico + 2 tablas
8. Matriz de correlación	Heatmap y comparación agrupada vs intra-estación	2 gráficos + tabla
9. Conclusiones	Nueve hallazgos con su acción recomendada	Tabla resumen

Ejecutable con `python examples/03_eda.ipynb` desde Jupyter o VS Code, seleccionando el intérprete de `.venv`. Las dependencias añadidas durante la sesión —`matplotlib`, `ipykernel`, `nbformat`, `nbconvert`— están instaladas en ese entorno pero **no declaradas** en `pyproject.toml`; si el notebook va al repositorio del equipo habría que añadirlas.

05 Investigación de modelos y esquema: estado parcial

Interrupción por límite de sesión

Se lanzaron dos workflows con 17 agentes en total. Diez completaron su trabajo; siete fallaron al agotarse el límite de la sesión, incluidos los tres jueces y las dos síntesis finales. Los resultados que siguen provienen de los agentes que sí terminaron y están respaldados por mediciones reales sobre los datos, pero no pasaron por la fase de evaluación cruzada prevista.

Lo que sí se midió

Un agente ejecutó backtesting temporal real con partición de 38 días de entrenamiento y 7 de validación, calculando WAPE por estación y promediando. Otro auditó la integridad del dataset con verificación independiente. Los resultados más relevantes:

- **El generador sintético no modela festivos.** En el rango hay dos festivos colombianos —7 de agosto y 17 de agosto— y la demanda esos días fue prácticamente idéntica a la de un día normal del mismo día de semana (ratios 0,974 y 0,988). Un festivo real en Bogotá se parecería a un domingo. Conclusión: no vale la pena construir un calendario de festivos.
- **El cociente fin de semana / entre semana es multiplicativo y plano.** Esto *contradice* lo que concluyó el EDA visual: medido por separado en pico AM, pico PM y valle de mediodía, el cociente es casi idéntico en los tres para cada estación. El pico matutino no desaparece el fin de semana, solo se escala. Aparecen además tres arquetipos discretos de estación ($\approx 0,475$, $\approx 0,82$ y $\approx 1,26$).
- **El pronóstico de lluvia está recortado en cero.** `rain_mm` es cero en apenas el 0,1% de los periodos, pero `rain_forecast` lo es en el 36,3%. No es una copia ruidosa del valor real, tiene una asimetría estructural que la correlación de 0,823 esconde.
- **Ya hay 7 días de datos retenidos.** Los datos terminan el 2026-09-08 y hoy es 2026-09-17: el organizador retiene 673 periodos sin liberar. El pipeline debe asumir desde el primer día que el último dato disponible no es reciente.
- **El panel está desbalanceado.** Los 45 días cubren 6 semanas completas más 3 días sueltos: hay 7 domingos, lunes y martes pero solo 6 de los demás días. Con lag semanal se pierde la primera semana, dejando unos 5 ciclos útiles para entrenar.

Advertencias operativas sobre el propio SDK

Un agente revisó el código del cliente y encontró dos comportamientos que conviene conocer antes de montar el pipeline:

- `client.download()` escribe el archivo y después lo borra con `unlink()` si el hash no coincide. Si el dataset cambia entre la descarga y la verificación, se pierde el archivo bueno que ya se tenía. Conviene descargar a una ruta temporal y mover solo tras validar.
- `_get()` no implementa reintentos ni backoff, así que un 429 o un 5xx transitorio —previsible si muchos equipos consultan a la vez desde GitHub Actions— aborta la ejecución.

Qué falta

Las recomendaciones de modelos no llegaron a consolidarse: faltó la evaluación por los tres jueces (precisión, operación y robustez ante drift) y la síntesis final con el ranking, el plan de features priorizado y la política de reentrenamiento. Lo mismo ocurrió con el esquema de base de datos: hay tres diseños completos e independientes y una crítica adversaria, pero no el DDL consolidado. Ambos workflows son

reanudables desde caché sin repetir el trabajo ya hecho.

06 Pendientes

Pendiente	Detalle	Estado
Artifact del EDA	Publicar el informe visual con datos faltantes, mapa, correlaciones y modelos sugeridos. Los datos ya están exportados a JSON y la paleta validada.	Bloqueado por la síntesis de modelos
Síntesis de modelos	Reanudar el workflow para obtener el ranking, el plan de features y la política de reentrenamiento.	Reanudable desde caché
Esquema de base de datos	Consolidar los tres diseños en un DDL único. Solo diseño: no se ejecuta contra Supabase.	Reanudable desde caché
Dependencias del notebook	Declarar matplotlib, ipykernel, nbformat y nbconvert en pyproject.toml si el notebook va al repositorio del equipo.	Decisión pendiente
Verificar el hallazgo contradictorio	El agente auditor sostiene que no hay interacción hora × tipo de día, solo un factor multiplicativo. El notebook afirma lo contrario. Conviene resolverlo antes de diseñar las features.	Requiere comprobación

Documento generado automáticamente como registro de la sesión de trabajo del 16 y 17 de septiembre de 2026. Los datos citados provienen de la ejecución real del notebook sobre 51.840 observaciones descargadas de la API pública del reto.